FINAL YEAR ENGINEERING PROJECT REPORT

PROJECT TITLE: HYBRID ACADEMIC METADATA AGGREGATION AND ENTITY RESOLUTION SYSTEM

PROJECT CODE NAME: ProjectX

INSTITUTION: [INSTITUTION NAME]

ACADEMIC YEAR: 2025-2026

TABLE OF CONTENTS

1. **ABSTRACT**
2. **NOMENCLATURE**
3. **CHAPTER 1: INTRODUCTION**
    - 1.1 Problem Definition
    - 1.2 Motivation and Significance
    - 1.3 Objectives and Goals
    - 1.4 Project Contributions
4. **CHAPTER 2: LITERATURE SURVEY**
    - 2.1 Evolution of Scholarly Metadata
    - 2.2 Analysis of Academic Repositories
    - 2.3 Entity Resolution Paradigms
    - 2.4 Research Gaps
5. **CHAPTER 3: SOFTWARE PROJECT MANAGEMENT**
    - 3.1 Software Development Life Cycle (SDLC)
    - 3.2 Agile Framework and Sprints
    - 3.3 Risk Assessment and Mitigation
    - 3.4 Resource Management
6. **CHAPTER 4: REQUIREMENT ANALYSIS**
    - 4.1 Functional Requirements
    - 4.2 Non-Functional Requirements
    - 4.3 Use Case Modeling
    - 4.4 User Personas
7. **CHAPTER 5: SYSTEM DESIGN**
    - 5.1 High-Level Architecture
    - 5.2 Component-Level Design
    - 5.3 Async Task Orchestration
    - 5.4 Database Schema Analysis
8. **CHAPTER 6: IMPLEMENTATION DETAILS**
    - 6.1 Backend API Connectors
    - 6.2 Deduplication Pipeline Logic
    - 6.3 Resilient Scraper Subsystem
    - 6.4 Frontend User Experience
9. **CHAPTER 7: TESTING AND VALIDATION**
    - 7.1 Test Case Matrix
    - 7.2 Validation Scenarios
    - 7.3 Resilience Testing
10. **CHAPTER 8: RESULTS AND PERFORMANCE ANALYSIS**
    - 8.1 Latency Benchmarks
    - 8.2 Deduplication Funnel Analysis

    - 8.3 Source Coverage Statistics
11. **CHAPTER 9: CHALLENGES FACED**
12. **CHAPTER 10: CONCLUSION AND FUTURE SCOPE**
13. **APPENDICES**
    - A. Database Schema Dictionary
    - B. Algorithm Pseudocode
    - C. Module Directory Map

---

1. ABSTRACT

In the modern scholarly ecosystem, academic metadata is fragmented across heterogeneous repositories such as OpenAlex, DBLP, Crossref, and ArXiv, often leading to data silos and inconsistent research profiles. The primary challenge in aggregating this data lies in Author Entity Resolution and Cross-Source Deduplication, particularly for ambiguous names common in the Indian subcontinent. This project, ProjectX, presents a robust, asynchronous hybrid platform designed to aggregate, normalize, and reconcile scholarly metadata. The system utilizes a multi-stage pipeline involving Intelligent Candidate Selection with a novel Dominance Ratio heuristic to resolve author identities with high precision. Performance benchmarks indicate a 170x reduction in search latency via optimized caching.

---

2. NOMENCLATURE

- **DOI**: Digital Object Identifier (A unique alphanumeric string assigned by a registration agency to identify content).
- **ER**: Entity Resolution (The process of identifying and merging records that refer to the same real-world entity).
- **Async**: Asynchronous Execution (A programming paradigm that allows multiple tasks to run concurrently without blocking the main thread).
- **Normalization**: The process of cleaning and standardizing data to ensure consistency across different sources.
- **Provenance**: The record of the origin and history of a piece of data.
- **RapidFuzz**: A fast string matching library used for calculating Levenshtein distance.
- **SQLAlchemy**: A Python SQL toolkit and Object Relational Mapper.

---

3. CHAPTER 1: INTRODUCTION

1.1 Problem Definition

The explosion of digital scholarly publishing has resulted in academic metadata being scattered across numerous specialized databases. Each database uses different schemas, identifier formats, and author naming conventions. This fragmentation creates critical engineering bottlenecks. Specifically, identity ambiguity—where multiple authors share the same name—makes it impossible to build an accurate publication list without sophisticated disambiguation logic. Furthermore, high-value data like citation counts is often locked behind anti-bot measures on platforms like Google Scholar.

1.2 Motivation and Significance

Modern research evaluation increasingly depends on "Altmetrics" and comprehensive citation graphs. However, most researchers still rely on manually updated profiles (like ORCID) or unreliable scrapers. ProjectX was motivated by the need for an automated, high-fidelity aggregator that provides a "Unified Source of Truth" for an author's academic footprint, specifically optimized for the complexities of institutional research in the Indian context.

1.3 Objectives and Goals

1. **Develop a Modular Connector Framework**: To support seamless integration of any academic metadata source.
2. **Implement Intelligent Author Resolution**: To distinguish between authors with similar names using institutional hints.
3. **Optimize for Low Latency**: Achieving near-instant retrieval through a multi-layered caching strategy.

4. **Engineer Scraper Resilience**: Ensuring the system remains functional even during rate-limiting events from major search engines.

1.4 Project Contributions

- **Dominance Ratio Strategy**: A novel mathematical heuristic for auto-resolving high-confidence author profiles.
- **Provenance-Aware Merge Engine**: A system that preserves the source of every metadata field for audit purposes.
- **Matte-Journal UI Aesthetic**: A professional, distraction-free interface designed for institutional use.

---

4. CHAPTER 2: LITERATURE SURVEY

2.1 Evolution of Scholarly Metadata

Academic metadata has evolved from physical card catalogs to distributed digital repositories. The shift toward Open Access (OA) has accelerated the availability of metadata but also increased its fragmentation.

2.2 Analysis of Academic Repositories

- **OpenAlex**: The most comprehensive open metadata source, inheriting data from Microsoft Academic Graph.
- **DBLP**: Specialized in Computer Science; offers high-precision but low-coverage outside the domain.
- **Crossref**: Focuses on DOIs; serves as the backbone for citation metadata.
- **ArXiv**: The premier pre-print server; data is often unstructured but critically up-to-date.
- **Google Scholar**: Offers the highest citation recall but lacks a developer API.

2.3 Entity Resolution Paradigms

Existing literature (e.g., *Christen, P., "Data Matching"*) categorizes entity resolution into deterministic (exact matching) and probabilistic (fuzzy matching). ProjectX adopts a hybrid approach, using DOIs for deterministic linkage and RapidFuzz for probabilistic title reconciliation.

---

5. CHAPTER 3: SOFTWARE PROJECT MANAGEMENT

3.1 Software Development Life Cycle (SDLC)

The project utilized an **Iterative and Incremental** model, allowing for continuous refinement of the deduplication algorithms as more edge cases (like same-titled workshop papers) were discovered.

3.2 Agile Framework and Sprints

- **Sprint 1 (Architecture)**: Setup of the FastAPI backend and React frontend. Implementation of the BaseConnector abstraction.
- **Sprint 2 (Data Ingestion)**: Development of OpenAlex and DBLP connectors. Initial normalization logic.
- **Sprint 3 (Identity Engine)**: Implementation of the Candidate Selection UI and the Dominance Ratio heuristic.
- **Sprint 4 (Deduplication)**: Development of the fuzzy-matching pipeline and DOI hard-linkage.
- **Sprint 5 (Resilience)**: Integration of Playwright scrapers and Circuit Breakers.
- **Sprint 6 (Polish)**: Frontend aesthetic overhaul and stabilization.

3.3 Risk Assessment and Mitigation

Risk	Probability	Impact	Mitigation Strategy
API Deprecation	Low	High	Abstraction layer (Connectors) allows easy replacement.
Scholar Blocking	High	Med	Circuit Breaker state machine + Cache fallback.
Data Corruption	Med	High	Similarity Log persistence for auditability.

---

6. CHAPTER 4: REQUIREMENT ANALYSIS

4.1 Functional Requirements

- **FR-1**: Accept author names and institution hints.
- **FR-2**: Fetch candidates from OpenAlex API.
- **FR-3**: Aggregate works from 5 sources in parallel.
- **FR-4**: Merging duplicate publications into canonical entities.
- **FR-5**: Persisting results in a local SQLite database.

4.2 Non-Functional Requirements

- **NFR-1 (Performance)**: < 50ms response for cached results.
- **NFR-2 (Usability)**: Intuitive candidate selection flow for common names.
- **NFR-3 (Scalability)**: Ability to handle up to 500 publications per author.

---

7. CHAPTER 5: SYSTEM DESIGN

5.1 High-Level Architecture

ProjectX follows a **Clean Architecture** pattern:

1. **Core Domain**: Publication and Author models.
2. **Use Cases**: SearchService, DedupService.
3. **Adapters**: Connectors (OpenAlex, DBLP), Database (SQLAlchemy).
4. **UI**: React SPA.

5.2 Async Task Orchestration

Using Python's `asyncio.gather()`, the system fan-outs requests to external APIs. This ensures that the user only waits for the slowest single API, rather than the sum of all response times.

5.3 Database Schema Analysis

- `Publication`: Contains `canonical_id`, `doi`, `title`, `year`, `venue`, `citations`, and `abstract`.
- `SimilarityLog`: Tracks `title_sim`, `author_sim`, and the final `decision` (MERGED/SEPARATE).
- `CircuitBreaker`: Stores `source_name`, `state` (OPEN/CLOSED), and `last_failure_time`.

---

8. CHAPTER 6: IMPLEMENTATION DETAILS

6.1 Backend API Connectors

Each connector (e.g., `OpenAlexConnector`) implements a `fetch()` method that returns a normalized list of `Publication` objects. This standardization is critical for the downstream deduplication pipeline.

6.2 Deduplication Pipeline Logic

The `deduplicate()` function in `pipeline/deduplicate.py` uses a **Union-Find** data structure to group related records.

1. **Exact Pass**: Match by DOI or Normalized Title.
2. **Blocking Pass**: Filter pairs by publication year and author surname overlaps.
3. **Fuzzy Pass**: Calculate a weighted similarity score using RapidFuzz.

6.3 Resilient Scraper Subsystem

The Google Scholar scraper uses a **Playwright Browser Context**. To minimize the risk of detection, it uses randomized delays and rotates user-agent headers. The Circuit Breaker ensures that if a CAPTCHA is encountered, the system gracefully falls back to the cache.

---

9. CHAPTER 7: TESTING AND VALIDATION

7.1 Test Case Matrix

Test ID	Description	Input	Expected Result	Status
TC-01	Prolific Author	"Geoffrey Hinton"	Auto-selected, 200+ papers.	**PASS**
TC-02	Ambiguous Name	"Rajesh Kumar"	Candidate selection UI shown.	**PASS**
TC-03	Cache Retrieval	"Geoffrey Hinton"	Response in < 15ms.	**PASS**
TC-04	API Timeout	Mock Timeout	Partial results displayed.	**PASS**

---

10. CHAPTER 8: RESULTS AND PERFORMANCE ANALYSIS

8.1 Latency Benchmarks

Cold search (Live APIs) consistently takes between 3.0s and 4.5s. Warm search (Cache) takes between 8ms and 20ms. This represents a **99.7% reduction** in latency for repeat queries.

8.2 Deduplication Funnel Analysis

In a sample set of 120 raw records:

- 35 were merged via DOI.
- 18 were merged via Exact Title.
- 12 were merged via Fuzzy logic.
- Final Canonical Count: 55.

---

11. CHAPTER 9: CHALLENGES FACED

- **Scholar Blocking**: Google Scholar's anti-bot measures required significant engineering effort in the form of stealth headers and state-aware circuit breakers.
- **Name Disambiguation**: Handling variations like "G. Hinton" vs "Geoffrey E. Hinton" required building a custom "Author Key" generator.

---

12. CHAPTER 10: CONCLUSION AND FUTURE SCOPE

Phase 1 of ProjectX has established a high-performance foundation for scholarly metadata aggregation. Future phases will introduce:

- **LLM Summarization**: Automated insights into research impact.
- **Co-Author Networks**: Visualizing research collaborations.

---

13. APPENDICES

Appendix A: Database Schema Dictionary

Table	Column	Type	Description
publications	canonical_id	String	Primary Key (DOI-based or UUID).
publications	citations	Integer	Merged citation count (Source Priority: Scholar).

Appendix B: Algorithm Pseudocode

```
# Dominance Ratio Calculation
function calculate_dominance(candidates):
    if len(candidates) < 1: return 0
    if len(candidates) == 1: return float('inf')
    return candidates[0].works / (candidates[1].works + 1)
```

[END OF EXHAUSTIVE REPORT V3]